

PA: Product and Presentation

NoMisHub is a web platform designed to fight misinformation by letting users publish and discuss news, while fact-checkers (moderators) and administrators help ensure content is properly sourced and follows community policies. The platform aims to promote both “No-Mis-Information” and “No-Miss-Information”, combining reliable content access with an active community discussion space.

A9: Product

In this artifact, we will supply the needed credentials and commands to run the Docker image of our project. We will also show the final progress of our project reflected on the final state of our User Stories completion as well as a bit of insight into the workings of the website like, for example, how validation is applied.

NoMisHub is a web-based information system focused on the publication, discussion, and moderation of news content. The platform supports multiple user roles—guests, registered users, moderators, and administrators—each with distinct permissions and responsibilities. Its core functionality revolves around news creation and categorization, user interaction through comments and votes, and moderation mechanisms to handle reports and enforce platform rules.

The application is structured into well-defined modules, including authentication and user profile management, news management, comments and voting, notifications, moderation and administration, and static informational pages (such as About, Help, and FAQ). This modular organization reflects the system’s architecture and supports maintainability, scalability, and clear separation of concerns.

1. Installation

Release code:

Release: [<PA Tag>](#)

Docker command:

```
docker run -d --name lbaw2511 -p 8001:80 gitlab.up.pt:5050/lbaw/lbaw2526/lbaw2511
```

The application will be available at <http://localhost:8001>

2. Usage

2.1. Administration Credentials

Username	Password	Email
admin	testing123	admin@admin.com

2.2. User Credentials

Type	Username	Password	Email
regular account	regUser	testing123	reg@user.com

2.3. Moderator Credentials

Username	Password	Email
bob	testing123	bob@example.com

2.4. MailTrap and Google credentials

The credentials for accessing the Mailtrap and Google accounts are the same and are presented bellow:

Email	Password
lbaw2511@gmail.com	HrICJ5JE%36\$&v

3. Application Help

Help features have been implemented throughout the platform to guide users in performing key actions and understand the system's functionality:

1. Contextual help in forms:

From the **search interface** (`homepage.blade.php` and `allnews.blade.php`):

- Search bar includes a placeholder with search icon: `placeholder="🔍 Search..."`
- Filter menu provides labeled sections for Sort By, Verified status, Author, and Category
- Each filter input includes descriptive placeholders (e.g., "Username...", "Tag name...")

Expanded filter menu with all options visible:

From **tag management** (`tags.blade.php`):

- Clear instruction text: "Type a name below to create a new official tag."
- Input placeholder provides example: `placeholder="e.g., Technology"`
- Descriptive labels explain each field's purpose

Tag creation form with the help text visible:

2. Interactive guidance:

From **promotion forms** (`promotetoadmin.blade.php` and `promotetomoderator.blade.php`):

- Clear confirmation messages: "Are you sure you want to promote [username] to [role]? You can't reverse this action."
- Action consequences are explicitly stated before execution

One of the promotion confirmation pages:

From **report viewing** (`report.blade.php`):

- Structured information display with clear labels ("Reported user:", "Content:", "Acknowledged by:")
- Links to relevant pages (user profiles, reported content URLs)
- Status indicators show whether reports have been acknowledged

Report detail page with all sections visible:

3. Search and filter guidance:

The advanced search interface provides:

- Dropdown menu with sorting options ("Newest (Default)", "Most Upvoted")
- Checkbox for filtering verified content with visual indicator: "Show Verified Only  "
- Text inputs for precise filtering by author username or category name
- "Apply" and "Clear" buttons for managing filters

Filter dropdown menu fully expanded with all options:

4. User feedback and confirmation:

From `comments.js` and `moderator.js` :

- Validation alerts provide specific error messages ("Please write a comment", "Comment cannot be empty", "Please enter a valid duration between 1 and 168 hours")
- Success confirmations after actions (e.g., "Timeout applied successfully!")
- Deletion confirmations prevent accidental data loss ("Are you sure you want to delete this comment?")

From `tags.blade.php` :

- Warning messages for destructive actions: "Are you sure? This will remove the tag from all news articles."
- Success messages displayed after successful operations

One of the confirmation dialogs or success messages:

5. Permission-based UI:

The platform automatically adjusts available actions based on user authentication and role:

- Non-authenticated users see limited options (no create button, no follow buttons)
- Authenticated users see full interaction features (comment, vote, follow)
- Moderators see additional tools (checkmark toggle, timeout buttons)
- Administrators see management interfaces (tag management, report handling, user promotion)

The same page from different user perspective (guest vs authenticated vs moderator):

6. Dedicated information pages:

As mentioned in the document, the platform includes:

- **About page:** Explains NoMisHub's mission and key features.
- **MainFeatures page:** Presents the main features of the website.
- **Contacts page:** Provides ways to reach the developers.

These pages are accessible from the main navigation and provide comprehensive information about the platform's purpose and policies:

4. Input Validation

Client-side validation examples

Client-side validation is used to provide immediate feedback and prevent obviously invalid actions before a request is sent to the server.

1. Comment submission validation

From `comments.js` in the `submitComment` function:

```
function submitComment(newsId) {  
  const input = document.getElementById('comment-input-' + newsId);  
  const content = input.value.trim();  
  
  if (!content) {  
    alert('Please write a comment');  
    return;  
  }  
  // ... continues with fetch request  
}
```

Comment input field with the alert message "Please write a comment" appearing when user tries to submit an empty comment:

2. Comment editing validation

From `comments.js` in the `editComment` function:

```
const saveEdit = () => {
  const newContent = input.value.trim();

  if (!newContent) {
    alert('Comment cannot be empty');
    return;
  }

  if (newContent === currentContent) {
    cancelEdit();
    return;
  }
  // ... continues with AJAX request
}
```

Comment being edited with the validation alert appearing when attempting to save an empty edit:

3. Timeout duration validation

From `moderator.js` in the `submitTimeout` function:

```
function submitTimeout(userId) {
  const duration = document.getElementById('timeout-duration').value;
  const reason = document.getElementById('timeout-reason').value.trim();

  // Validate inputs
  if (!duration || duration < 1 || duration > 168) {
    alert('Please enter a valid duration between 1 and 168 hours.');
```

```
if (!reason) {
  alert('Please enter a reason for the timeout.');
```

```
  return;
}

const durationInt = parseInt(duration);
if (isNaN(durationInt)) {
  alert('Please enter a valid number for duration.');
```

```
  return;
}
// ... continues with timeout application
}
```

Timeout form with validation alerts for invalid duration or missing reason fields:

4. Permission-based UI updates

From `homepage.blade.php` and `allnews.blade.php` :

```
@auth
  <a href="{{ route('news.create') }}" class="create-news-btn" title="Create News">+</
@endauth
```

And in the category follow section:

```
@auth
  <button
    id="category-follow-btn"
    class="button {{ $isFollowingCategory ? 'button-unfollow' : 'button-follow' }}"
    data-following="{{ $isFollowingCategory ? '1' : '0' }}"
  >
    {{ $isFollowingCategory ? 'Unfollow' : 'Follow' }}
  </button>
@endauth
```

Two side-by-side screenshots - one as a guest user (no create button) and one as an authenticated user (more interactive elements):

5. HTML5 required attributes in forms

From `tags.blade.php` in the tag creation form:

```
<input
  type="text"
  name="name"
  placeholder="e.g., Technology"
  required
  style="flex: 1; padding: 10px; border: 1px solid #ccc; border-radius: 4px;">
```

Attempting to submit the tag creation form without entering a name, triggering the browser's built-in "Please fill out this field" message:

6. Confirmation dialogs for destructive actions

From `comments.js` in the `deleteComment` function:

```
function deleteComment(commentId) {
  if (!confirm('Are you sure you want to delete this comment?')) {
```

```
        return;
    }
    // ... continues with delete request
}
```

And from `tags.blade.php` for tag deletion:

```
<form action="{{ route('admin.tags.destroy', ['category' => $tag->name]) }}" method="POST"
        onsubmit="return confirm('Are you sure? This will remove the tag from all news articles');">
    @csrf
    @method('DELETE')
    <button type="submit" class="button">Delete</button>
</form>
```

Browser confirmation dialog appearing before deleting a comment or tag:

Server-side validation examples

Server-side validation is the authoritative validation layer and is responsible for ensuring correctness, security, and consistency.

1. User profile validation code

From `UserController.php` in the `update` method:

```
$validated = $request->validate([
    'username' => 'nullable|string|max:250|unique:users',
    'name' => 'nullable|string|max:250',
    'email' => 'nullable|email|max:250|unique:users',
    'description' => 'nullable|string|max:1000',
    'file' => 'nullable|image|mimes:jpeg,jpg,png,gif'
]);
```

This ensures that usernames and emails remain unique across the platform, descriptions don't exceed reasonable lengths, and uploaded profile images are of valid formats.

2. Password change validation code

From `UserController.php` in the `update` method:

```
// First, verify current password
$validator = Validator::make($request->all(), [
    'current_password' => [
        'required', function ($attribute, $value, $fail) {
            if (!Hash::check($value, Auth::user()->password)) {
                $fail('Current Password doesn\'t match');
            }
        },
    ],
],
);
```

```
// Then validate new password
$validated = $request->validate([
    'new_password' => 'required|min:8|confirmed'
]);
```

This two-step validation ensures users can only change their password if they know the current one, and the new password meets minimum security requirements.

3. Moderator timeout action validation code

From `UserController.php` in the `applyTimeout` method:

```
$request->validate([
    'duration_hours' => 'required|integer|min:1|max:168',
    'reason' => 'required|string|max:500'
]);
```

Additional validation examples from the platform:

4. User registration validation

From `RegisterController.php`:

```
$request->validate([
    'username' => 'required|string|max:250|unique:users',
    'name' => 'required|string|max:250',
    'email' => 'required|email|max:250|unique:users',
    'password' => 'required|min:8|confirmed'
]);
```

5. Report submission validation

From `ReportController.php` in the `store` method:

```
$request->validate([
    'content' => 'required|string|max:1000',
    'user_id' => 'required|integer|exists:users,id',
    'target_url' => 'url',
]);
```

This ensures reports contain meaningful content, are linked to valid users, and reference valid URLs.

6. Password recovery validation

From `UserController.php` in the `recoverPassword` method:

```
$request->validate([
    'password'=> 'required|min:8|confirmed',
]);

$validator = Validator::make($request->all(), [
    'code' => [
        'required', function ($attribute, $value, $fail) use ($code) {
            if ($value != $code) {
                $fail('Recovery Code doesn\'t match');
            }
        },
    ],
],
);
```

This validates that the recovery code matches the one sent via email before allowing password reset.

7. Email validation for password recovery

From MailController.php :

```
$validator = Validator::make($request->all(), [
    'email' => [
        'required', function ($attribute, $value, $fail) {
            if (is_null(User::where('email', $value)->first())) {
                $fail('Email not found');
            }
        },
    ],
],
]);
```

8. File upload validation

From FileController.php :

```
$validator = Validator::make($request->all(), [
    'file' => [
        function ($attribute, $value, $fail) use ($request) {
            // 1. File must exist
            if (!$request->hasFile('file')) {
                return $fail('File not found');
            }

            // 2. Upload type must be valid
            if (!$self::isValidType($request->type)) {
                return $fail('Unsupported upload type');
            }

            // 3. Extension must be valid
            $file = $request->file('file');
            $extension = $file->getClientOriginalExtension();

            if (!$self::isValidExtension($request->type, $extension)) {
                return $fail('Unsupported upload extension');
            }
        },
    ],
]);
```

5. Check Accessibility and Usability

Unfortunately, due to limitations either of the browser or of the website used for the report, the documents aren't properly formatted when converted to PDF format.

- [Accessibility Report](#)
- [Usability Report](#)

6. HTML & CSS Validation

- [HTML Validation Report for a news page \(/news/{id}\).](#)
- [CSS Validation Report for news.css](#)

7. Revisions to the Project

Relatively to artifact 2 (A2: Actors and User stories), there weren't many differences between the initial plans and the final result. The user stories have stayed largely the same throughout the whole project. Some user stories, as presented bellow, weren't able to be implemented, but they consisted, mainly of [Innovation] User Stories (US39) and non-obligatory User Stories (not underlined on the requirements - US36). Possibly because of overconfidence and lack of proper means to calculate the needed time. The only major difference was, following the feedback given by the professor, collapsing US30 and US31 into a single User Story, since they had overlapping objectives.

Relatively to artifact 7 (A7: Web Resources Specification), both the Modules and Permissions remain the same as, in our perspective, they were fairly well defined at the time and the expected roles remained the same.

8. Implementation Details

8.1. Libraries Used

During the development of this project, we used two frameworks:

- Laravel: for the development of the backend of the website, ranging from the accesses to the database to the returning of the proper pages to the user. Example: [NewsController.php](#)
- Bootstrap: for complementing the CSS of our website. It's use is more noticeable in forms, in which it was used to center and organise the elements of the page. It is also used in other parts of the website, but not as extensive. Example: [createnews.blade.php](#)

We used no other external libraries or frameworks.

8.2 User Stories

8.2.1. User

Identifier	Name	Module	Priority	Team Members	State
US01	Top News	M02	Prototype	Tomás Silva, António Pais	100%
US02	News Item	M02	Prototype	Tomás Silva, Hugo Alves, António Pais	100%
US03	Exact Match Search	M02	Prototype	Tomás Silva, António Pais	100%
US04	Full-text Search	M02	Prototype	Tomás Silva, António Pais	100%
US05	Recent News	M02	Product	Tomás Silva, António Pais	100%
US07	Search over Multiple Attributes	M02	Product	Tomás Silva	100%
US08	Search Filters	M02	Product	Tomás Silva	100%
US09	Form Help	Product	M01, M02, M03, M04	Ana Patrício, Hugo Alves, Pedro Pais	100%
US10	Error Messages	M01, M02, M03, M04	Product	Ana Patrício, Hugo Alves, Pedro Pais	100%
US11	About Us	M05	Product	António Pais	100%
US12	Main Features	M05	Product	António Pais	100%
US13	Contacts	M05	Product	António Pais	100%
US14	See Checkmark	M02	Innovation	Tomás Silva, António Pais	100%

Table 2: User user stories.

8.2.2. Non-Authenticated User (Guest)

Identifier	Name	Module	Priority	Team Members	State
US15	Login	M01	Prototype	Ana Patrício, Hugo Alves	100%
US16	Registration	M01	Prototype	Ana Patrício, Hugo Alves	100%
US17	Recover Password	M01	Product	Ana Patrício, Hugo Alves	100%

Table 3: Guest user stories.

8.2.3. Authenticated User

Identifier	Name	Module	Priority	Team Members	State
US18	Logout	M01	Prototype	Ana Patrício, Hugo Alves	100%

Identifier	Name	Module	Priority	Team Members	State
US19	View Profile	M01	Prototype	Ana Patrício, Hugo Alves	100%
US20	Manage Profile	M01	Prototype	Ana Patrício, Hugo Alves	100%
US21	User News Feed	M02	Prototype	António Pais	100%
US22	News Item Details	M02	Prototype	António Pais	100%
US23	Publish News Item	M02	Prototype	António Pais, Hugo Alves	100%
US24	View Others' Profiles	M01	Prototype	Ana Patrício, Hugo Alves, António Pais	100%
US25	Manage News Item	M02	Prototype	António Pais, Hugo Alves	100%
US26	Vote on Content	M03	Product	António Pais	100%
US27	Comment on News Item	M03	Product	António Pais	100%
US28	Delete Account	M01	Product	Ana Patrício, Hugo Alves	100%
US29	Manage Profile Picture	M01	Product	Ana Patrício, Hugo Alves	100%
US30	Notifications	M01	Product	António Pais, Hugo Alves	100%
US31	Mark Notification as Read	M01	Product	António Pais, Hugo Alves	100%
US32	Comment Details	M03	Product	Tomás Silva, António Pais	100%
US33	Reputation	M01	Product	Ana Patrício, António Pais, Hugo Alves	100%
US34	View News Tags	M02	Product	Tomás Silva, Hugo Alves	100%
US35	Follow/Unfollow Users and Tags	M01, M02	Product	Hugo Alves, Ana Patrício	100%
US36	Suggest Tags	M02	Product	Hugo Alves, Tomás Silva	50%
US37	Manage Comments	M03	Product	Hugo Alves, António Pais	100%
US38	Report User or Content	M04	Product	Hugo Alves	100%
US39	Respond to Promotion	M04	Innovation	Hugo Alves	0%

Table 4: Authenticated User user stories.

8.2.4. Administrator

Identifier	Name	Module	Priority	Team Members	State
US40	Manage User Accounts	M04	Prototype	Hugo Alves, António Pais	100%
US41	Manage User Content	M04	Product	Hugo Alves, António Pais	100%
US42	Manage Tags	M04	Product	Hugo Alves, Tomás Silva	100%
US43	Respond to Reports	M04	Product	Hugo Alves	100%

Identifier	Name	Module	Priority	Team Members	State
US44	Promote to Administrator	M04	Innovation	Hugo Alves	100%
US45	Promote to Moderator	M04	Innovation	Hugo Alves	100%

Table 5: Administrator user stories.

8.2.5. Moderator

Identifier	Name	Module	Priority	Team Members	State
US46	Add Checkmark	M02, M04	Innovation	Tomás Silva, Pedro Pais	100%
US47	Time-out Users	M04	Innovation	Hugo Alves, Pedro Pais	100%

Table 6: Moderator user stories.

A10: Presentation

In this artifact we will make a brief presentation of our final product.

1. Product presentation

NoMisHub is a community-oriented platform that enables users to engage with news content in an active and moderated environment. Visitors can freely browse published news, while registered users can contribute by publishing articles, commenting, voting, and following other users. These interactions are complemented by a notification system that keeps users informed about activity related to their content.

The delivered version showcases the complete usage flow of the platform, from user authentication and content publication to moderation actions and administrative oversight. Built with Laravel and PostgreSQL, the application demonstrates how community interaction, moderation tools, and system-enforced rules work together to support informed discussion and reduce the spread of misinformation.

To run our project the following steps are needed:

1. Run the following Docker command:

```
docker run -d --name lbaw2511 -p 8001:80 gitlab.up.pt:5050/lbaw/lbaw2526/lbaw2511
```

The application will be available at <http://localhost:8001>

2. Log-in with one of the accounts presented on A9 or navigate the website as a guest user.

2. Video presentation

Since the deadline for the video presentation is only on the 31st December 2025, but the delivery of this document is on the 23rd December 2025 and, as permitted by professor Sérgio Nunes, we will leave this area empty as the video is not yet available.

Revision history

Changes made to the first submission:

GROUP2511, 21/12/2025

- Ana Catarina Barbosa Patrício, up202107383@fe.up.pt (editor)
- António Pedro Alves Pais, up202305444@fe.up.pt
- Hugo Ribeiro Alves, up202305395@fe.up.pt
- Tomás Pereira da Silva, up202307796@fe.up.pt

Comments